

|                                                                                                                                                    |                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <p><b>FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES</b></p> | Departamento: Ciencias de la Computación                                                                                                             |
|                                                                                                                                                    | Carrera: Software Asignatura: Análisis y Diseño de Software<br>NRC: 30723                                                                            |
|                                                                                                                                                    | Apellidos y nombres del estudiante:<br>Llumiyinga Moreno Jerson Stiven<br><br>Montufar Calvache Luis Alejandro<br><br>Morillo Mosquera Ariel Esteban |

Sangolquí, a 09 de julio de 2026

## PLAN DE PRUEBAS DE CALIDAD (QA) - Finca Jirah

### 1. Resumen Ejecutivo y Alcance

Este documento establece la estrategia y el diseño de pruebas unitarias y de integración para el backend y lógica de negocio del proyecto "Finca Jirah", una aplicación agrícola PWA Offline-First.

#### Alcance de las pruebas:

- Lógica de Negocio (`lib/services`): `AgricultorService`, `AuthService`, `PesajeService`, `ClasificacionService`.
- Lógica de Sincronización y Patrones de Diseño (`lib/adapters`, `lib/state`, `lib/observers`, `lib/services/calculo`): `OfflinePayloadAdapter`, `SyncState`, `SyncObserver`, `CalculadoraCosecha`.
- Reglas de dominio puras (Cálculo de mermas, validación de cédulas, encriptación).

#### Fuera de alcance:

- UI/Componentes de React y hooks de presentación.
- Pruebas E2E (End-to-End) en navegador.

### 2. Entorno y Herramientas

- **Framework de Pruebas: Vitest** (Recomendado por su integración nativa con arquitecturas modernas basadas en Vite/Next.js y TypeScript).
- **Aserciones y Espías:** API integrada de Vitest (compatible con Jest).
- **Mocking de Base de Datos: `vitest-mock-extended`** para generar un Mock estricto y seguro de tipos (Type-safe) del `PrismaClient`. No se realizarán conexiones a bases de datos reales durante las pruebas unitarias.
- **Cobertura de Código:** `Instambul (v8)` integrado en Vitest, apuntando a un mínimo de 85% de

cobertura en ramas críticas (lib/services).

### 3. Matriz de Trazabilidad

| Módulo        | Descripción                        | ID Requisito | Clases Asociadas                                                                                        | Estado Cobertura |
|---------------|------------------------------------|--------------|---------------------------------------------------------------------------------------------------------|------------------|
| Acceso        | Autenticación y Recuperación       | CU-01        | AuthService                                                                                             | Pendiente        |
| Perfil        | Configuración de Usuario           | CU-02        | PerfilService (por implem.)                                                                             | Pendiente        |
| Agricultores  | Gestión del Padrón de Trabajadores | CU-03        | AgricultorService                                                                                       | Pendiente        |
| Catálogos     | Campañas, Lotes, Compradores       | CU-04        | CatalogoService, Repo                                                                                   | Pendiente        |
| Transacciones | Cosechas y Offline-First (Core)    | CU-05        | PesajeService, ClasificacionService, CalculadoraCosecha, OfflinePayloadAdapter, SyncState, SyncObserver | Pendiente        |
| Dashboard     | Métricas de Productividad          | CU-06        | ReportesService (Dashboard)                                                                             | Pendiente        |

### 4. Diseño Granular de Casos de Prueba

#### CU-01: Acceder al Sistema

| ID Caso   | Nombre                                  | Precondiciones              | Datos de Entrada                  | Pasos de Ejecución                            | Resultado Esperado                                  | Estado    |
|-----------|-----------------------------------------|-----------------------------|-----------------------------------|-----------------------------------------------|-----------------------------------------------------|-----------|
| TC-01.1.1 | Iniciar sesión (Flujo Normal)           | Usuario registrado y activo | email y password correctos        | 1. Llamar AuthService.login(email, password). | Devuelve token JWT y datos básicos del usuario.     | Pendiente |
| TC-01.1.2 | Iniciar sesión (Credenciales inválidas) | Usuario registrado          | email válido, password incorrecto | 1. Llamar AuthService.login(email, password)  | Lanza Excepción: "Correo o contraseña incorrectos". | Pendiente |

| ID Caso   | Nombre                                    | Precondiciones               | Datos de Entrada           | Pasos de Ejecución                                      | Resultado Esperado                                     | Estado    |
|-----------|-------------------------------------------|------------------------------|----------------------------|---------------------------------------------------------|--------------------------------------------------------|-----------|
|           |                                           |                              |                            | ).                                                      |                                                        |           |
| TC-01.1.3 | Iniciar sesión (Cuenta inactiva)          | Usuario con isActive = false | email y password correctos | 1. Llamar AuthService.login(email, password).           | Lanza Excepción: "Su cuenta se encuentra desactivada". | Pendiente |
| TC-01.2.1 | Recuperar contraseña (Flujo Normal)       | Usuario existe en DB         | email registrado           | 1. Llamar AuthService.generateTokenRecuperacion(email). | Genera token válido por 1 hora y envía correo.         | Pendiente |
| TC-01.2.2 | Recuperar contraseña (Correo inexistente) | Correo no registrado         | email no registrado        | 1. Llamar AuthService.generateTokenRecuperacion(email). | Retorna null/silencios o por seguridad.                | Pendiente |
| TC-01.2.3 | Recuperar contraseña (Token caducado)     | Token generado hace >1 hora  | Token expirado             | 1. Llamar validación de token.                          | Lanza Excepción: "El enlace expiró".                   | Pendiente |

#### CU-02: Configurar Perfil de Usuario

| ID Caso   | Nombre                              | Precondiciones      | Datos de Entrada                      | Pasos de Ejecución                                       | Resultado Esperado                                    | Estado    |
|-----------|-------------------------------------|---------------------|---------------------------------------|----------------------------------------------------------|-------------------------------------------------------|-----------|
| TC-02.1.1 | Actualizar datos (Flujo Normal)     | Usuario autenticado | telefono válido                       | 1. Llamar PerfilService.actualizarDatos(id, {telefono}). | Los datos se actualizan, campos sensibles bloqueados. | Pendiente |
| TC-02.2.1 | Modificar contraseña (Flujo Normal) | Usuario autenticado | claveActual válida, nuevaClave segura | 1. Llamar PerfilService.modificarContraseñas             | Se actualiza el hash en la DB correctamente.          | Pendiente |

| ID Caso   | Nombre                                | Precondiciones      | Datos de Entrada       | Pasos de Ejecución                                              | Resultado Esperado                              | Estado    |
|-----------|---------------------------------------|---------------------|------------------------|-----------------------------------------------------------------|-------------------------------------------------|-----------|
|           |                                       |                     |                        | a(id, actual, nueva).                                           |                                                 |           |
| TC-02.2.2 | Modificar contraseña (Actual errónea) | Usuario autenticado | claveActual incorrecta | 1. Llamar PerfilService.modificarContraseña(id, actual, nueva). | Lanza Excepción por contraseña actual inválida. | Pendiente |
| TC-02.3.1 | Modificar preferencias (Flujo Normal) | Dispositivo con red | tema = "Oscuro"        | 1. Llamar guardado de preferencias.                             | Se guarda en base de datos.                     | Pendiente |
| TC-02.3.2 | Modificar preferencias (Sin red)      | Dispositivo offline | tema = "Oscuro"        | 1. Detectar estado offline.2. Guardar preferencia local.        | Se guarda en localStorage sin invocar API.      | Pendiente |

### CU-03: Gestionar Agricultores

| ID Caso   | Nombre                              | Precondiciones     | Datos de Entrada              | Pasos de Ejecución                         | Resultado Esperado                      | Estado    |
|-----------|-------------------------------------|--------------------|-------------------------------|--------------------------------------------|-----------------------------------------|-----------|
| TC-03.1.1 | Crear agricultor (Flujo Normal)     | Sin duplicados     | Nombres, Cédula válida, Email | 1. Llamar AgricultorService.crear(input).  | Retorna el objeto user y _dev_password. | Pendiente |
| TC-03.1.2 | Crear agricultor (Cédula duplicada) | Cédula ya existe   | Cédula existente              | 1. Llamar AgricultorService.crear(input).  | Lanza Excepción: CEDULA_EN_USO.         | Pendiente |
| TC-03.2.1 | Consultar agricultor (Flujo Normal) | Agricultores en DB | Query = nombre o cédula       | 1. Llamar AgricultorService.buscar(query). | Devuelve listado de coincidencias.      | Pendiente |
| TC-03.3.1 | Editar                              | Agricultor         | Nombres                       | 1. Llamar                                  | Retorna                                 | Pendiente |

| ID Caso   | Nombre                                     | Precondiciones         | Datos de Entrada      | Pasos de Ejecución                                   | Resultado Esperado                      | Estado    |
|-----------|--------------------------------------------|------------------------|-----------------------|------------------------------------------------------|-----------------------------------------|-----------|
|           | agricultor (Flujo Normal)                  | existente              | modificados           | AgricultorService.editar(id, input).                 | agricultor actualizado, Cédula intacta. |           |
| TC-03.3.2 | Editar agricultor (Correo duplicado)       | Correo asignado a otro | Email de otro usuario | 1. Llamar AgricultorService.editar(id, input).       | Lanza Excepción: EMAIL_EN_USO.          | Pendiente |
| TC-03.4.1 | Desactivar agricultor (Flujo Normal)       | Agricultor existente   | ID válido             | 1. Llamar AgricultorService.desactivar(id, adminId). | Soft-delete exitoso (isActive = false). | Pendiente |
| TC-03.4.2 | Desactivar agricultor (Auto-desactivación) | Admin logueado         | ID = adminId          | 1. Llamar AgricultorService.desactivar(id, adminId). | Lanza Excepción: SELF_DEACTIVATION.     | Pendiente |

#### CU-04: Configurar Catálogos de la Finca

| ID Caso   | Nombre                        | Precondiciones         | Datos de Entrada                     | Pasos de Ejecución                      | Resultado Esperado                      | Estado    |
|-----------|-------------------------------|------------------------|--------------------------------------|-----------------------------------------|-----------------------------------------|-----------|
| TC-04.1.1 | Registrar Campaña (Éxito)     | No existe código en DB | codigo = "102026", comprado = "ID-1" | 1. Llamar CampanaService.create(input). | Retorna campaña con taringaBase a 1.70. | Pendiente |
| TC-04.1.2 | Registrar Campaña (Duplicado) | Campaña ya existe      | codigo = "102026"                    | 1. Llamar CampanaService.create(input). | Lanza Excepción: CODIGO_DUPLICADO.      | Pendiente |
| TC-04.1.3 | Consultar Campañas            | Campañas creadas       | N/A                                  | 1. Llamar CampanaService.listar().      | Devuelve array de campañas.             | Pendiente |

| ID Caso   | Nombre                          | Precondiciones      | Datos de Entrada      | Pasos de Ejecución                              | Resultado Esperado                 | Estado    |
|-----------|---------------------------------|---------------------|-----------------------|-------------------------------------------------|------------------------------------|-----------|
| TC-04.1.4 | Editar Campaña (Éxito)          | Campaña sin pesajes | Modificación general  | 1. Llamar CampanaService.actualizar(id, input). | Actualiza datos de la campaña.     | Pendiente |
| TC-04.1.5 | Editar Campaña (Bloqueo tara)   | Campaña con pesajes | taraBase modificada   | 1. Llamar CampanaService.actualizar(id, input). | Lanza Excepción: TARA_BLOQUEADA.   | Pendiente |
| TC-04.1.6 | Cerrar Campaña (Éxito)          | Sin pesajes PENDING | ID campaña            | 1. Llamar CampanaService.cerrar(id).            | Cambia estado a cerrada.           | Pendiente |
| TC-04.1.7 | Cerrar Campaña (Sync pendiente) | Pesajes PENDING     | ID campaña            | 1. Llamar CampanaService.cerrar(id).            | Lanza Excepción: SYNC_PENDIENTE.   | Pendiente |
| TC-04.2.1 | Registrar Lote (Éxito)          | Nombre único en DB  | nombre = "Lote Norte" | 1. Llamar LoteService.crear(input).             | Lote persistido correctamente.     | Pendiente |
| TC-04.2.2 | Registrar Lote (Duplicado)      | Lote ya existe      | codigo duplicado      | 1. Llamar LoteService.crear(input).             | Lanza Excepción: CODIGO_DUPLICADO. | Pendiente |
| TC-04.2.3 | Consultar Lotes                 | Lotes registrados   | N/A                   | 1. Llamar LoteService.listar().                 | Devuelve listado de áreas.         | Pendiente |
| TC-04.2.4 | Editar Lote                     | Lote existente      | hectareas = 10        | 1. Llamar LoteService.actualizar(id, input).    | Retorna lote modificado.           | Pendiente |
| TC-04.2.5 | Inactivar Lote (Éxito)          | Lote sin campaña    | ID lote               | 1. Llamar LoteServ                              | Soft-delete exitoso.               | Pendiente |

| ID Caso   | Nombre                          | Precondiciones          | Datos de Entrada                                 | Pasos de Ejecución                                             | Resultado Esperado                            | Estado    |
|-----------|---------------------------------|-------------------------|--------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------|-----------|
|           |                                 | activa                  |                                                  | <code>ice.eliminar(id).</code>                                 |                                               |           |
| TC-04.2.6 | Inactivar Lote (En uso)         | Campaña activa asociada | ID lote                                          | 1. Llamar <code>LoteService.eliminar(id).</code>               | Lanza Excepción: <code>LOTE_EN_USO</code> .   | Pendiente |
| TC-04.3.1 | Registrar Comprador (Éxito)     | RUC único               | <code>ruc = "17900",</code><br><code>tipo</code> | 1. Llamar <code>CompradorService.crear(input).</code>          | Crea comprador con perfil de tolerancia.      | Pendiente |
| TC-04.3.2 | Registrar Comprador (Duplicado) | RUC existente           | <code>ruc</code><br><code>duplicado</code>       | 1. Llamar <code>CompradorService.crear(input).</code>          | Lanza Excepción: <code>RUC_DUPLICADO</code> . | Pendiente |
| TC-04.3.3 | Consultar Compradores           | Compradores activos     | N/A                                              | 1. Llamar <code>CompradorService.listar()</code> .             | Devuelve listado comercial.                   | Pendiente |
| TC-04.3.4 | Editar Comprador                | Comprador existente     | <code>contacto</code><br><code>nuevo</code>      | 1. Llamar <code>CompradorService.actualizar(id, input).</code> | Retorna datos actualizados.                   | Pendiente |
| TC-04.3.5 | Eliminar Comprador              | Comprador existente     | ID comprador                                     | 1. Llamar <code>CompradorService.eliminar(id).</code>          | Soft-Delete exitoso.                          | Pendiente |

#### CU-05: Registrar transacciones en campo (CORE y Patrones)

| ID Caso   | Nombre                                | Precondiciones         | Datos de Entrada                                  | Pasos de Ejecución                                     | Resultado Esperado                               | Estado    |
|-----------|---------------------------------------|------------------------|---------------------------------------------------|--------------------------------------------------------|--------------------------------------------------|-----------|
| TC-05.1.1 | Registrar pesaje bruto (Flujo Normal) | Campaña y lote activos | <code>pesoBrutoKg,</code><br><code>gavetas</code> | 1. Llamar <code>PesajeService.registrar(input).</code> | Retorna pesaje con <code>pesoNetoKg</code> base. | Pendiente |

| ID Caso   | Nombre                                                | Precondiciones           | Datos de Entrada       | Pasos de Ejecución                                                          | Resultado Esperado                              | Estado    |
|-----------|-------------------------------------------------------|--------------------------|------------------------|-----------------------------------------------------------------------------|-------------------------------------------------|-----------|
| TC-05.1.2 | Registrar pesaje bruto (Sin red)                      | Dispositivo offline      | pesoBrutoKg, gavetas   | 1. Llamar adapter offline.                                                  | Guarda en SQLite con estado PENDING.            | Pendiente |
| TC-05.2.1 | Registrar clasificación (Flujo Normal)                | Pesaje bruto existe      | Categorías desglosadas | 1. Llamar ClasificacionService.register(input).                             | Guarda clasificación validando suma.            | Pendiente |
| TC-05.2.2 | Registrar clasificación (Descuadre supera $\pm 4\%$ ) | Diferencia > 4%          | Suma incoherente       | 1. Instanciar CalculadoraCosecha.2. Llamar ClasificacionService.register(). | Lanza Excepción exigiendo confirmación forzada. | Pendiente |
| TC-05.3.1 | Registrar ajuste de comprador                         | Clasificación existe     | kg_rechazados          | 1. Llamar ajuste en servicio.                                               | Recalcula fruta efectiva.                       | Pendiente |
| TC-05.4.1 | Sincronizar datos (Flujo Normal)                      | Datos PENDING, Hay red   | Estado SYNC            | 1. Ejecutar OfflinePayloadAdapter.adaptToPostgres().                        | Respuesta 200 OK, cambia estado local a SYNCED. | Pendiente |
| TC-05.4.2 | Sincronizar datos (Falla 500)                         | Datos PENDING, Falla API | Estado SYNC            | 1. Ejecutar sincronización.                                                 | Mantiene PENDING e inicia reintentos (backoff). | Pendiente |

#### CU-06: Analizar Productividad

| ID Caso   | Nombre                               | Precondiciones  | Datos de Entrada | Pasos de Ejecución               | Resultado Esperado                  | Estado    |
|-----------|--------------------------------------|-----------------|------------------|----------------------------------|-------------------------------------|-----------|
| TC-06.1.1 | Dashboard interactivo (Flujo Normal) | Existen pesajes | N/A              | 1. Llamar servicio de Dashboard. | Devuelve agregaciones estadísticas. | Pendiente |
| TC-06.1.2 | Filtrar por fechas (Flujo)           | Fechas válidas  | Rango correcto   | 1. Llamar filtrado de            | Devuelve métricas del               | Pendiente |



| ID Caso   | Nombre                            | Precondiciones     | Datos de Entrada  | Pasos de Ejecución               | Resultado Esperado                        | Estado    |
|-----------|-----------------------------------|--------------------|-------------------|----------------------------------|-------------------------------------------|-----------|
|           | Normal)                           |                    |                   | fechas.                          | rango.                                    |           |
| TC-06.1.3 | Filtrar por fechas (Incongruente) | Fecha inicio > fin | Rango inválido    | 1. Llamar filtrado de fechas.    | Lanza Excepción de rango inválido.        | Pendiente |
| TC-06.1.4 | Filtrar por Lote físico           | Lote válido        | ID lote           | 1. Llamar filtrado por lote.     | Devuelve rendimiento de plantas del lote. | Pendiente |
| TC-06.1.5 | Filtrar por Campaña/Fu nda        | Campaña válida     | Filtro campaña    | 1. Llamar filtrado por campaña.  | Devuelve métricas específicas.            | Pendiente |
| TC-06.1.6 | Generar reporte exportable        | Datos procesados   | Formato PDF/Excel | 1. Llamar generación de reporte. | Retorna buffer del archivo.               | Pendiente |

## 5. Criterios de Aceptación y Suspensión

### Criterios de Aceptación

- **100% de los Casos de Prueba Críticos (CU-03 y CU-05) ejecutan exitosamente (Pass).**
- Cobertura de pruebas (Code Coverage) igual o mayor al 85% para los servicios en `lib/services`.
- Todos los mocks de base de datos no presentan fugas (leaks) que intenten conexiones reales.
- Se valida la mutabilidad dinámica del Patrón Strategy (`CalculadoraCosecha.setEstrategiaMargen`).

### Criterios de Suspensión

- Si se detecta que las pruebas dependen de una instancia local o remota de base de datos en ejecución (violación de principio de aislamiento unitario).
- Fallas de compilación en TypeScript que bloqueen la ejecución de Vitest.
- Si las reglas de negocio del CU-05 (Pesajes y tolerancias) fallan en más de 2 casos, se suspenden las pruebas de integración hasta corrección arquitectónica.